# Research Report

Suppapat Korsurat

August 2020

## 1 Notation and Background

Suppose that we have N i.i.d. data points, $x_i$, with the corresponding latent variable, $z_i$. The latent variable $z_i$ will come from the prior distribution, $p(\mathbf{z})$. In addition, the data, $x_i$ will come from the likelihood distribution, $p_\theta(\mathbf{x}|\mathbf{z})$ where $\theta$ is the parameter of the generative network.

### 1.1 Reweighted Expectation Maximization

[DP19]
The goal of the Reweighted Expectation Maximization (REM) is to learn both the posterior distribution $p_\theta(\mathbf{z}|\mathbf{x})$ and the parameter of the network, $\theta$.

Similar to the Expectation Maximization algorithm, REM has two steps, E-Step and M-Step.

For the E-Step, we will set the proposal distribution, $q_\phi(\mathbf{z})$ to be equal to $p_{\theta_t}(\mathbf{z}|\mathbf{x})$ The proposal distribution is a distribution that used for learning $\theta$.

The purpose of the M-Step is to find the optimal $\theta_{t+1}$ at the iteration $t+1$ that maximize the $\mathrm{L}(\theta) = \sum_{i=1}^{N} E_{p_{\theta_t}(z_i|x_i)}\big[\log p_\theta(x_i, z_i) - \log p_{\theta_t(z_i|x_i)}\big] = \sum_{i=1}^{N} E_{r_{\eta_t}(z_i|x_i)}\left[\frac{\mathbf{W}(x_i,z_i;\theta_t,\eta_t)\log p_\theta(x_i,z_i)}{E_{r_{\eta_t}(z_i|x_i)}[\mathbf{W}(x_i,z_i;\theta_t,\eta_t)]}\right]$ where $r_{\eta_t}(z_i|x_i)$ is a proposal distribution.

For the proposal distribution, we need a hyperporposal distribution to learn the parameter of the proposal, $\eta_t$. By moment-matching algorithm, the hyperproposal will be defined as a Gaussian distribution with parameters $\mu_{it}$ and $\Sigma_{it}$ where $\mu_{it} \approx \sum_{k=1}^{K} \alpha_{it}^k z_i^{(k)}$ and $\Sigma_{it} \approx \sum_{k=1}^{K} \alpha_{it}^k \big[(z_i^{(k)} - \mu_{it})(z_i^{(k)} - \mu_{it})^T\big]$

### 1.2 Annealed Importance Sampling

[Nea98]
Suppose that we would like to calculate the expectation of some function of z where z comes from the posterior distribution, denoted as $f_0(z)$. Moreover, we have intermediate distribution, $f_1(z)$ to $f_n(z)$, which they will assist us on sampling z from $f_0(z)$.

For the $f_n(z)$, we will use a prior distribution. For each distribution, we must have $p_j(z)$ which proportional to $f_j(z)$. For the intermediate distributions, we can calculate from $p_j(z) = p_0(z)^{\beta_j} p_n(z)^{1-\beta_j}$ where $1 = \beta_0 > \beta_1 > ... > \beta_n = 0$. Since we focus on the posterior expectation, the prior distribution can be unnomalized.

$T_j(z, z^*)$ is a transitional probability, calculated by traditional methods such as Gibbs sampling update. $T_j(z, z^*)$ is a probability that moving to the state $z^*$ when the current state is $z$. The subscription j stands for the corresponding intermediate distribution.

**Algorithm 1:** Annealed Importance Sampling

```
for i = 1 to N do
    Simulate z_{n-1} ~ f_n(z)
    Simulate z_{n-2} from z_{n-1} using T_{n-1}(z_{n-1}, z_{n-2})
    ...
    Simulate z_0 from z_1 using T_1(z_1, z_0)
end
Result: N samples from f_0(z)
```

For each iteration of the algorithm, we will use $z_0$ as a $z^(i)$, the simulated data point. For each simulated data point, it will have corresponding weight, calculated by $w^{(i)} = \frac{p_{n-1}(z_{n-1})}{p_n(z_{n-1})} \frac{p_{n-2}(z_{n-2})}{p_{n-1}(z_{n-2})} \cdots \frac{p_1(z_1)}{p_2(z_1)} \frac{p_0(z_0)}{p_1(z_0)}$.

### 1.3 Hamiltonian Monte Carlo (HMC)

[Nea12]
Hamiltonian Monte Carlo is a algorithm used for simulating the samples from the posterior distribution by using the first-order gradient information and auxiliary momentum variables.

For parameter of interest (or position) $z$, the HMC will introduce the momentum variable $r$ which will be independently drawn from the standard Gaussian distribution.

We can rewrite the probability distribution into the canonical distribution form, $P(z, r) = \frac{1}{C} \exp(-U(z)/T) \exp(-K(r)/T)$. $U(z)$ is a potential energy function which we will use to express the posterior and $K(r)$ is a kinetic energy function which specify by the distribution of the momentum variable. In addition, $U(z) + K(r) = H(z, r)$ which the function $H(z, r)$ is called Hamiltonian.

**Algorithm 2:** Hamiltonian Monte Carlo

```
for i = 1 to N do
    ## Simulate Step
    simulate the momentum variable: r ~ N(0,1)
    let z_start = z_{i-1} and r_start = r
    r = r + (ε/2) ▽_z U(z)
    for j = 1 to L do
        z = z + εr
        if j ≠ L then r = r − ε ▽_z U(z)
    end
    r = r − (ε/2) ▽_z U(z)
    z = −z
    ## Evaluate Step
    With probability α = min(1, exp(H(z,r) − H(z_start, p_start)))
    set z_n = z_{n-1} and r_n = r_{n-1}
end
Result: N Simulated data points from the posterior distribution by using Hamiltonian Monte Carlo.
```

Most of the time, we will set N to be a large number, then eliminate some of the first part of the result as it is a burn-in.

## 2 Experiments

In these experiments, I will use the simulated data by first simulating the prior $z_i$ from the Gaussian Distribution, then use the $z_i$ to simulate the corresponding data point $x_i$ from the Gaussian Distribution.

**Algorithm 3:** Simulating the data.

```
for i = 1 to 10,000 do
    Simulate z_i ~ N(μ, σ²_prior)
    Simulate x_i ~ N(θ_1 z_i + θ_2, σ²_data)
end
Result: Simulated 10,000 data points.
```

I will assume that the parameters of the prior distribution ($\mu$ and $\sigma^2_{prior}$), as well as the $\theta_1$, $\theta_2$ and $\sigma^2_{data}$ are known.

### 2.1 The number of the particles

In this experiment, I would like to know that if the number of the particles used in the Reweighted Expectation Maximization (REM) algorithm is small, can the generative network of the algorithm still converge to the actual value of the $\theta$.

The simulated data used in this experiment are from the Gaussian Distribution, $x_i \sim N(z_i, 0.01)$ where I have set the $\theta_1$ equals to 1, $\theta_2$ equals to 0, and $z_i$ is a latent variable which comes from the prior distribution, defined as a Gaussian distribution $z_i \sim N(0, 100)$

For the generative network, I have set various initial values of the $\theta_1$, denoted as $\hat{\theta}_1$. The result shows in figure 1.

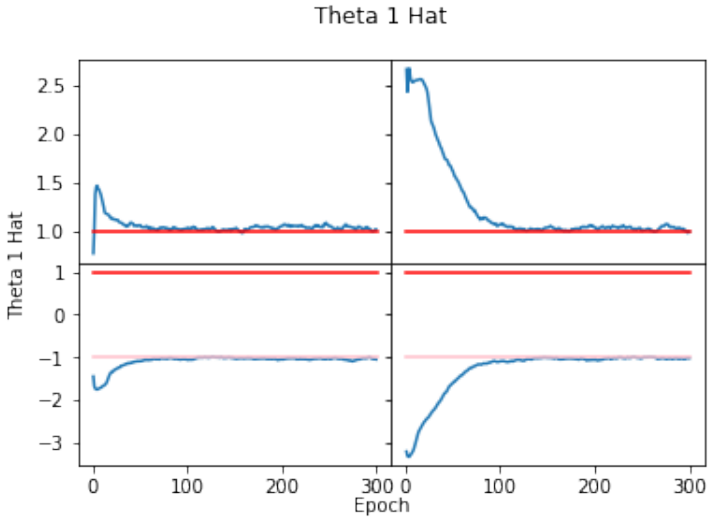


Figure 1: The result from the model. The red line is the exact value of the $\theta_1$. The upper plots are the result when the initial value of the $\hat{\theta}_1$ is a positive number. The lower plots result when the initial value of the $\hat{\theta}_1$ is a negative number.

According to figure 1, there is a problem when we initial the $\hat{\theta}_1$ to be a negative number. The final result tends to converge to the -1, which is the negative value of the exact $\theta_1$, instead of 1. To solve the problem above, I will let the generative model learn $\log(\hat{\theta}_1)$ instead.

#### 2.1.1 Only $\log(\hat{\theta}_1)$ is updated

The simulated data used in this experiment are from $x_i \sim N(2.5z_i, 0.01)$, where I have set the $\theta_1$ equals to 2.5. For the $z_i$, I use the Gaussian distribution with mean 0 and variance 100. For the generative network, I have initial the $\theta_1$ value in a log form, denoted as $\log(\hat{\theta}_1)$.

To verify the assumption that we would like to investigate, I have set $\log(\hat{\theta}_1) = -5$. Also, we ran the algorithm for 500 epochs.

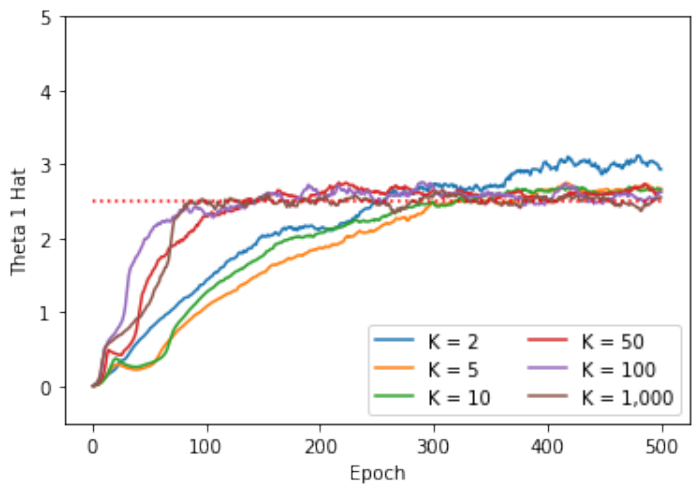


Figure 2: The result of the $\hat{\theta}_1$ when changing the size of the particles. The red dotted line is the value of $\theta_1$, which is the exact value.

The result from the figure 2 shows that when the number of the particles is too small (less than ten particles), the algorithm tends to not converge to the exact value. When the amount of the particles is large enough, 50

for this case, the algorithm converges to the exact value.

### 2.1.2 Both $\log(\hat{\theta}_1)$ and $\hat{\theta}_2$ are updated

In the previous experiments, I have set both $\theta_2$ and $\hat{\theta}_2$ to 0 and not updated it. I will do another experiment involving both $\theta_1$ and $\theta_2$ and let them be updated simultaneously.

Data are simulated from the from the Gaussian distribution $x_i \sim N(2z_i + 2, 0.01)$. For the $z_i$, I use the same prior distribution.

For the model, I have initialized the parameters $\log(\hat{\theta}_1) = -5$ and $\hat{\theta}_2 = -1$. The result shows in figure 3.

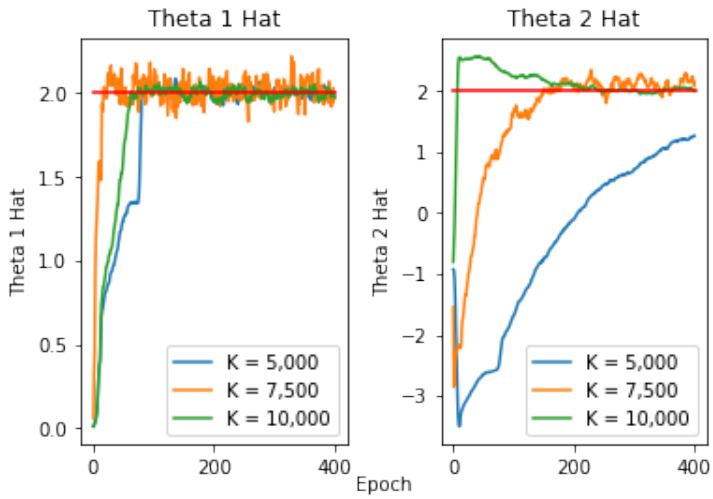


Figure 3: The result of the $\hat{\theta}_1$ and $\hat{\theta}_2$ when changing the size of the particles. The red lines are the value of $\theta_1$ and $\theta_2$.

The result shows that if the number of the particles is not large enough, the $\hat{\theta}_2$ will not converge to the exact value while there is no problem with the $\hat{\theta}_1$.

## 2.2 Method for finding the hyperproposal

For the traditional reweighted expectation maximization algorithm, the hyperproposal is a Gaussian distribution, where its parameters can be found by the moment-matching algorithm. In this experiment, I will adapt the Annealed Importance Sampling into the REM to see where the performance of the algorithm can be improved or not?

Here is the algorithm that I use in this experiment, which I use annealed importance sampling to learn the parameter $\eta$ in the proposal distribution.

**Algorithm 4:** Reweighted Expectation Maximization with Annealed Importance Sampling

* Need to initialize
(1) Parameters of the model, $\theta$.
(2) Parameters of the proposal distribution, $\eta$.
**Input:** Data $\mathbf{x}$
**for** *Epoch* $t$ = *1,2,...* **do**
  Draw a batch from the data, $\mathbf{x}_i$ where $i = 1, 2, ..., B$
  **for** *Data* $i$ = *1,2,...,B* **do**
    `// Update` $\theta$ `in the model.`
    Simulate $z_i^{(1)}, z_i^{(2)}, ...z_i^{(K)} \sim r_{\eta_t}(z_i^{(k)}|x_i)$ `//` $r_{\eta_t}(z_i^{(k)}|x_i)$ `is a proposal distribution.`
    Compute the importance weight $w_{it}^{(k)} = \frac{p_{\theta_t}(z_i^{(k)}, x_i)}{r_{\eta_t}(z_i^{(k)}|x_i)} = \frac{p(z_i^{(k)})p_{\theta_t}(x_i|z_i^{(k)})}{r_{\eta_t}(z_i^{(k)}|x_i)}$
    Calculate $\alpha_{it}^{(k)} = \frac{w_{it}^{(k)}}{\sum_{k=1}^{K} w_{it}^{(k)}}$
    `// Update` $\eta$`, parameter of the proposal distribution, by using Annealed Importance Sampling.`
    **for** $m$ = *1,2,...,K* **do**
      * The $f_n$ is a prior distribution, $p(\mathbf{z})$.
      The $f_0$ is a product of the $f_n$ and the likelihood $p_{\theta_t}(\mathbf{x}|\mathbf{z})$.
      `// Annealed Importance Sampling Framework`
      Draw $z_m^{(D-1)}$ from $f_n$ `// D is the number of the intermediate distribution.`
      Draw $z_m^{(D-2)}$ from $z_m^{(D-1)}$ by using $T_{D-1}$
      ...
      Draw $z_m^{(0)}$ from $z_m^{(1)}$ by using $T_1$
      Let $z_i^{(m)} = z_m^{(0)}$ with the corresponding weight $v_{it}^{(m)}$
    **end**
    Calculate $\beta_{it}^{(m)} = \frac{v_{it}^{(m)}}{\sum_{m=1}^{K} v_{it}^{(m)}}$
  **end**
  Compute $\bigtriangledown_\eta L(\eta)$ and update $\eta$ by using Adam.
  Compute $\bigtriangledown_\theta L(\theta)$ and update $\theta$ by using Adam.
**end**

For the transitional probability in the Annealed Importance Sampling, I will use the Hamiltonian Monte Carlo (HMC).

# References


[DP19] Adji B. Dieng and John Paisley. *Reweighted Expectation Maximization*. 2019. arXiv: `1906.05850 [stat.ML]`.

[Nea12] Radford M. Neal. *MCMC using Hamiltonian dynamics*. 2012. arXiv: `1206.1901 [stat.CO]`.

[Nea98] Radford M. Neal. *Annealed Importance Sampling*. 1998. arXiv: `physics/9803008 [physics.comp-ph]`.